

NAGARE

Holonomy Learning with Projection-Gated Global Pooling

Kyberszittya / HyMeKo Research Extraction

2026-07-02

Abstract

NAGARE is an experimental Rust implementation of local holonomy-based learning with quaternion periodic features, global pooling, Clifford error metrics, and projection-gated updates. The current extraction targets small generated point-set tasks such as moons, spiral, and xor. The strongest current result is a fitted rank-6 projection gate that improves the difficult spiral few-shot/noisy/missing stress row while keeping microsecond-scale CPU forward passes and a small parameter count.

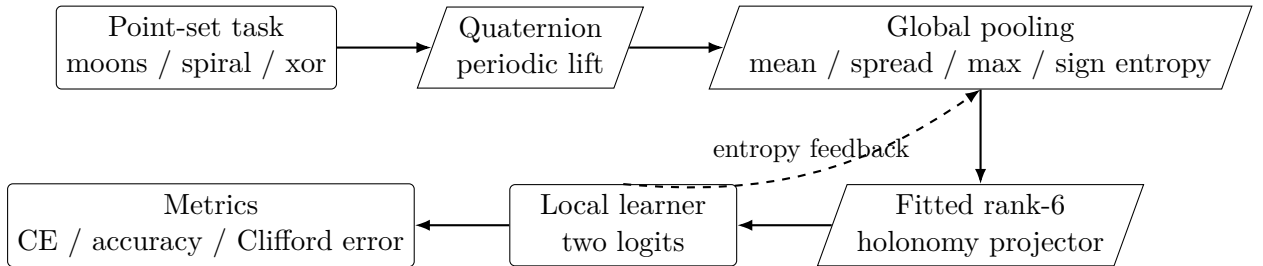
1 Positioning

NAGARE is not presented as a general replacement for backpropagation. It is a research artifact for testing whether structured local updates can be useful when the input is represented through geometry, periodicity, and holonomy.

The repository intentionally reuses HyMeKo primitives where the logic is not NAGARE-specific. Quaternion rotor operations and Clifford probability error are provided by `hymeko_clifford` through a path dependency during incubation.

2 Architecture

Each generated sample is a small point set. The point set is lifted into quaternion periodic features, pooled globally, optionally projected through a holonomy-informed subspace, and used by a compact local learner.



3 Feature and Gate Definition

The quaternion periodic feature lift emits seven channels per point:

$$[x, y, r, \text{rotated}_x, \text{rotated}_y, \text{holonomy}_w, \text{holonomy}_z].$$

Global pooling produces four statistics per lifted channel:

$$\text{mean, spread, max, sign entropy.}$$

The fitted projection gate estimates a rank-6 basis from class centroids and rotor/holonomy channel groups, then applies alpha mixing:

$$\phi' = \alpha P(\phi) + (1 - \alpha)\phi, \quad \alpha = 0.72.$$

This is the key shift from scalar multiplication to projection. The gate shapes the feature state by subspace selection instead of merely rescaling the update.

4 Main Result

The following table summarizes the fitted-projection extraction baseline. The backprop-like timings come from the original HyMeKo workspace comparison; the standalone extraction reproduces the local losses exactly after aligning rotor conventions and pinning `rand = 0.9.2`.

Task	Local acc	Local loss	Local $\mu\text{s}/\text{sample}$	Backprop-like $\mu\text{s}/\text{sample}$	Speedup
moons	1.000	0.001513	5.673	139.984	24.7x
spiral	1.000	0.004486	5.776	136.064	23.6x
xor	1.000	0.006714	4.178	136.299	32.6x

The fitted projection stress learner reports 228 parameters. The original backprop-like baseline reports 2836 parameters.

5 Gate Ablation

The difficult spiral few-shot/noisy/missing row is the most informative stress case.

Gate	Accuracy	Loss
Scalar entropy	0.927	0.286730
Constant	0.938	0.269849
Fixed projection	0.906	0.324541
Fitted projection	0.938	0.250603

The fixed projector was algebraically closer to holonomy than scalar multiplication, but it was too narrow. The fitted projector is the first version that is both conceptually cleaner and empirically stronger on the hard row.

6 Repository Layout

Path	Purpose
<code>src/datasets.rs</code>	Generated moons, spiral, xor data and corruption modes.
<code>src/features.rs</code>	Quaternion periodic lift using <code>hymeko_clifford</code> rotors.
<code>src/pooling.rs</code>	Global mean/spread/max/sign-entropy pooling.
<code>src/projection.rs</code>	Fixed and fitted holonomy projection basis.
<code>src/learner.rs</code>	Local learner, gates, timing, stress ablation.
<code>src/metrics.rs</code>	Cross entropy and Clifford probability error.
<code>examples/toy_compare.rs</code>	Reproducible toy benchmark.
<code>reports/</code>	Raw JSON and markdown reports from the extraction.

7 Verification

The current repository was verified with:

```
cargo test
cargo clippy --all-targets -- -D warnings
cargo run --release --example toy_compare -- \
  --tasks moons,spiral,xor --n-train 192 --n-test 96 \
  --n-points 32 --epochs 50 --batch-size 32 --lr 0.05 \
  --seed 53 --out reports/standalone-seed53-toy-run.json
```

8 Next Steps

1. Add explicit `project_alpha_mix_forward` and `project_alpha_mix_backward` kernels.
2. Add finite-difference checks for projection gradients.
3. Add a fixture parser for parity against the seed-53 run.
4. Add graph/pgraph feature enrichment and structural generators.
5. Compare against PyTorch CPU/GPU only at batch sizes where GPU overhead is meaningful.

9 Caveat

The results are promising but narrow. Current evidence supports the statement that projection-gated local holonomy learning is fast and compact on these generated structured toy problems. It does not yet establish a general replacement for backpropagation.